

小车巡线

小车巡线

1. 快速上手
2. 硬件接线
3. 使用方法
4. 现象结果
5. 代码解释

`line_following_init` 函数

`trace_task` 模块

`app_motor` 模块

1. 快速上手

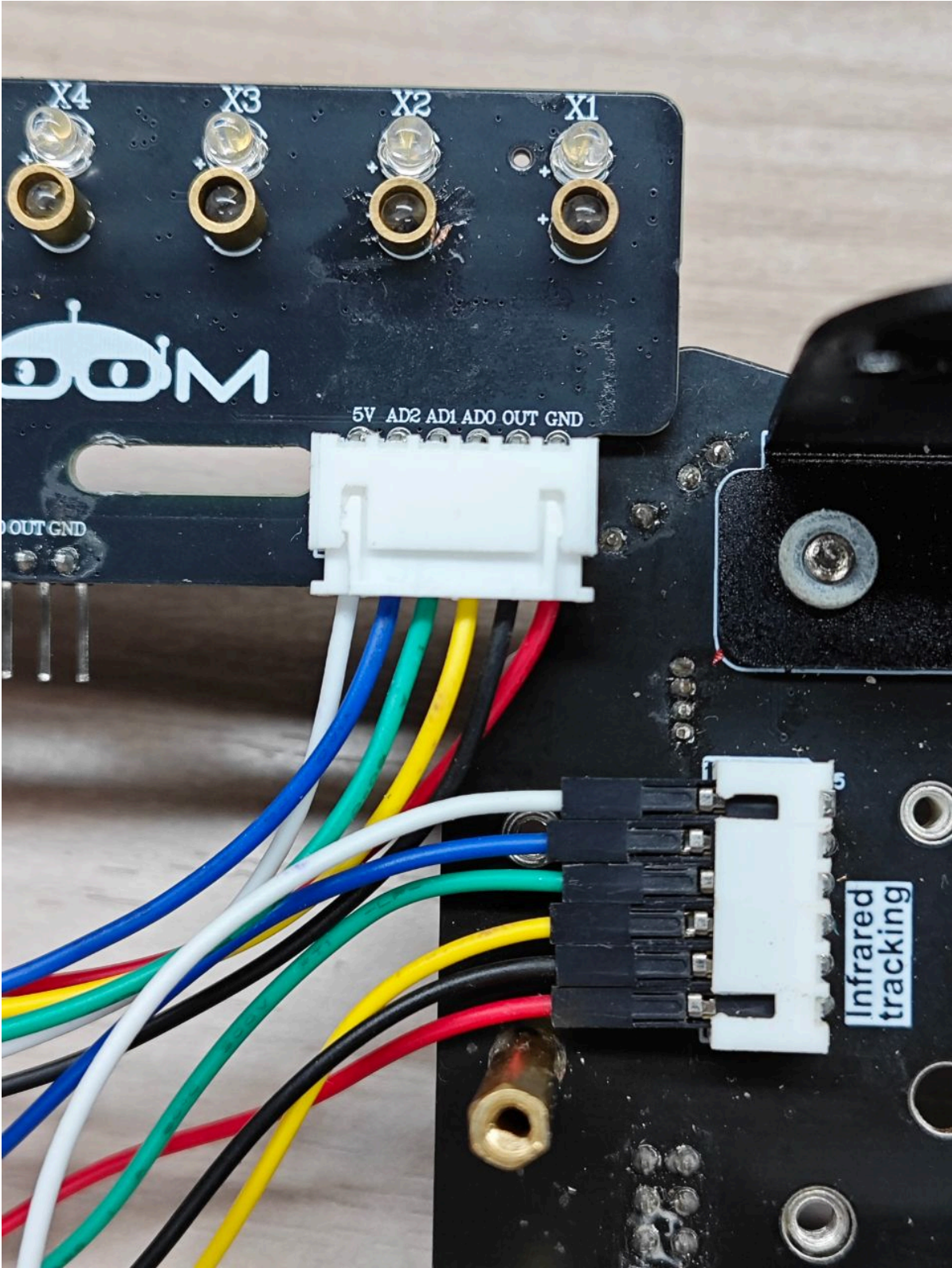
本教程讲解如何使用 STM32 主板在获取八路红外循迹传感器的数据之后，如何修改代码中的部分参数，使小车在用户自己的赛道场景中运行的效果更好。

教程中使用的硬件为亚博售卖的STM32开发板小车、八路红外循迹传感器、310电机、7.4V电池组。查看下方的接线图进行接线，烧录程序后，将小车放置在循迹赛道上，当传感器检测到黑线时，小车会自动解锁并开始巡线。如果巡线效果不佳，可以跳转到代码讲解章节，对PID参数进行修改，优化巡线效果。

此调优步骤需要用户具备一定的调试小车经验。非亚博的模块仅供参考。

2. 硬件接线

将STM32开发板小车翻转到底面，可以看到一个丝印 `Infrared tracking`，由这个口来连接我们的八路灰度模块。由于接口旁边没有具体引脚的丝印，所以可以根据图片中不同颜色的杜邦线来跟着接线。



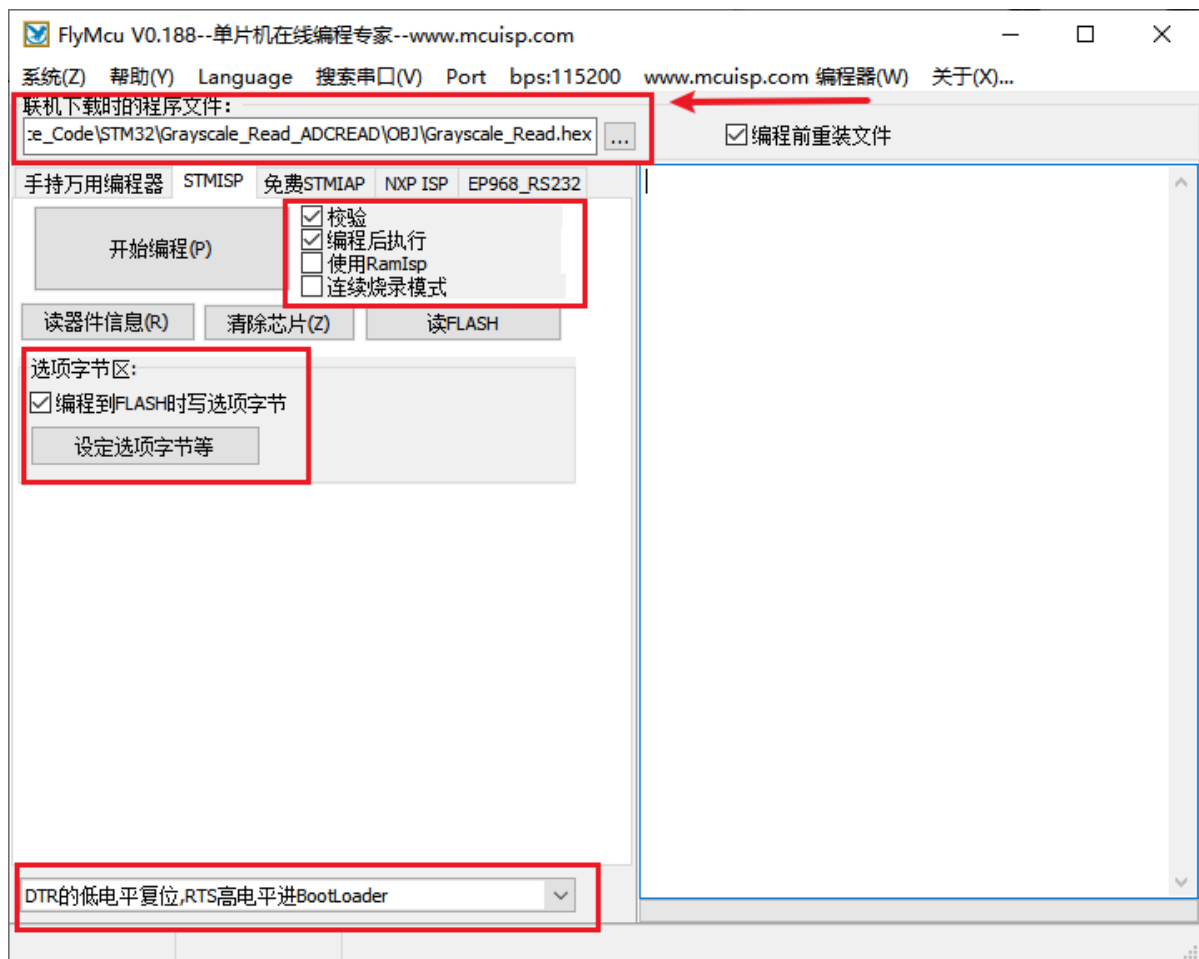
八路灰度巡线模块	STM32开发板小车
5V	5V
GND	GND
AD0	PF15
AD1	PF14
AD2	PF13
OUT	PG0

亚博售卖的八路灰度模块连接使用的线材为XH2.54转6pin杜邦线：

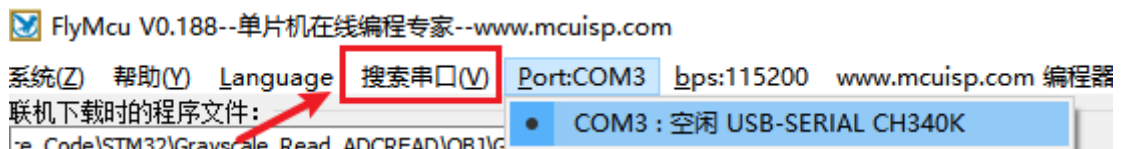


3. 使用方法

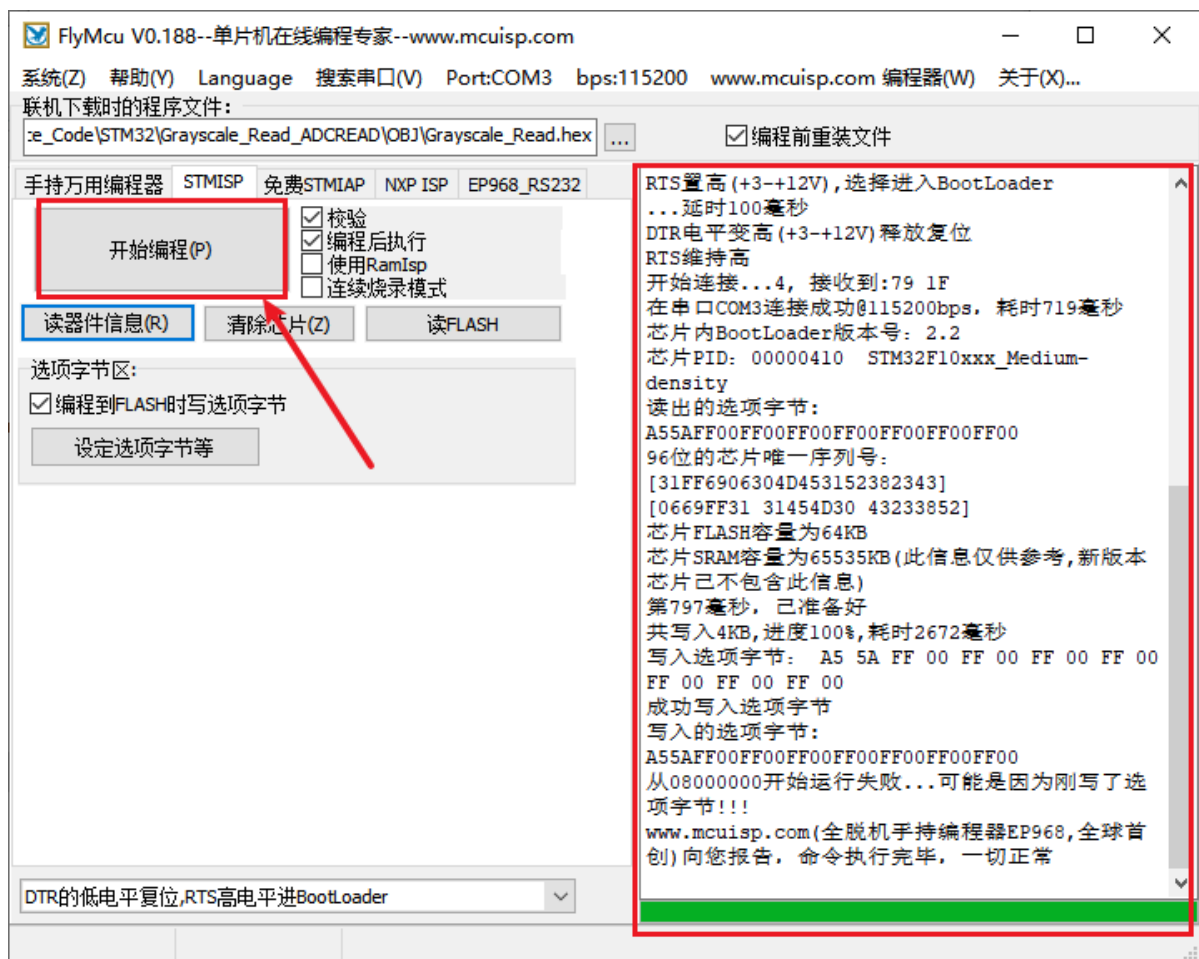
1. 打开FlyMcu烧录工具，程序文件选择代码工程里面的hex文件，其他红框框起来的地方需要与图片保持一致。



2. 将STM32与电脑连接，点击搜索串口，然后选择STM32属于的COM口。



3. 然后点击开始编程，程序开始被烧入到主板中，等待右边的进度条走完，也可以看到烧录成功的信息



4. 烧录完毕后，将小车放置与赛道上，打开开关，开始巡线。

4. 现象结果

上电之后，程序开始运行。此时如果八路灰度巡线模块上的八个LED灯都是全亮或者全灭，小车则不会动，防止因为被放在了错误的地图上乱跑。当巡线模块上的灯不一致时，小车开始巡线。

在要巡的线上的巡线模块LED灯，需要和线外的赛道相反，也就是说假如线上的LED灯是灭的，线外面的赛道上对应的LED灯需要亮起，才能正常巡线。

5. 代码解释

```
//#####  
// 重要配置参数 Important Configuration Parameters  
//#####  
// 将灰度巡线模块放在赛道上，观察对着线的灯，如果灯亮，数值为1，LINE_RAW_VALUE=1；如果灯灭，数值为0，LINE_RAW_VALUE=0  
// Place the grayscale line-following module on the track and observe the light facing the line. If the light is on, the value is 1, LINE_RAW_VALUE=1; if the light is off, the value is 0, LINE_RAW_VALUE=0  
#define LINE_RAW_VALUE 1  
//#####
```

main.c代码文件开头的地方有 `LINE_RAW_VALUE` 宏定义的设置。

- `LINE_RAW_VALUE`：需要将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则填 `LINE_RAW_VALUE=1`；如果灯灭，则数值为0，`LINE_RAW_VALUE=0`。


```
// USER/main.c
void line_following_init(line_following_t* controller) {
    controller->kp = 270.0f;    // 比例系数 | proportional gain
    controller->ki = 0.5f;      // 积分系数 | integral gain
    controller->kd = 50.0f;      // 微分系数 | derivative gain

    controller->base_speed = 450;    // 基础速度 | base speed
    controller->max_speed = 800;     // 最大速度 | maximum speed

    controller->sensor_weights[0] = -5.0f;
    controller->sensor_weights[1] = -4.0f;
    controller->sensor_weights[2] = -2.0f;
    controller->sensor_weights[3] = -1.0f;
    controller->sensor_weights[4] = 1.0f;
    controller->sensor_weights[5] = 2.0f;
    controller->sensor_weights[6] = 4.0f;
    controller->sensor_weights[7] = 5.0f;
}
```

line_following_init 函数

优化巡线效果重点修改项:

PID 参数

- **kp**: 调大 **kp** 可使响应更快, 但易震荡; 调小 **kp** 可减少震荡, 但响应会变慢且可能偏离路线。
- **ki**: 调大 **ki** 可更快消除稳态误差, 但易引起超调和震荡; 调小 **ki** 可降低超调风险, 但消除误差变慢甚至无法完全消除。
- **kd**: 调大 **kd** 可提升系统稳定性, 减少摆动和超调, 但过大会对噪声敏感并引发抖动; 调小 **kd** 可降低对噪声的敏感, 但抑制摆动能力减弱, 响应可能滞后或超调。

速度参数

- **base_speed**: 决定小车在平稳巡线时的前进速度, 影响整体效率和弯道响应。
- **max_speed**: 限制小车轮的最高转速, 防止小车加速偏差过大、过快。

灰度巡线模块权重参数

- **sensor_weights**: 修改灰度巡线模块上的权重, 第一个值指代模块最边缘的X1灯, 第二个值为X2灯。数值调大, 则在X1灯亮时反应更剧烈, 调小则平缓。

```
// APP/trace_task.c

bool check_sensors_safe(line_following_t* controller, uint16_t* sensor_values) {
    uint16_t first_value = sensor_values[0];
    //...
    return false;
}

float calculate_error(line_following_t* controller, uint16_t* sensor_values,
uint16_t line_raw_value) {
    float weighted_sum = 0.0f;
    int active_sensors = 0;
```

```

    //...
    return error;
}

float pid_control(line_following_t* controller, float error) {
    if (fabsf(error) < 0.6f) {
        error = 0.0f;
    }
    //...
    return output;
}

void differential_speed_control(line_following_t* controller, float pid_output,
                               int16_t* left_speed, int16_t* right_speed) {
    //...
}

void follow_line(line_following_t* controller, uint16_t* sensor_values,
                 uint16_t line_raw_value) {
    if (controller->motor_locked) {
        if (check_sensors_safe(controller, sensor_values)) {
            controller->motor_locked = false;
        } else {
            Ctrl_Speed(0, 0, 0, 0);
            return;
        }
    }

    float error = calculate_error(controller, sensor_values, line_raw_value);
    float pid_output = pid_control(controller, error);
    int16_t left_speed, right_speed;
    differential_speed_control(controller, pid_output, &left_speed,
&right_speed);

    Ctrl_Speed(left_speed, left_speed, right_speed, right_speed);
}

```

trace_task 模块

- `check_sensors_safe` : 检查所有灰度传感器是否都显示相同值，用于在启动时判断小车是否处于安全状态。
- `calculate_error` : 根据灰度传感器读取的值和预设权重，计算小车偏离线中心的误差值。
- `pid_control` : 执行PID控制算法，根据输入的误差值计算控制输出。
- `differential_speed_control` : 根据PID控制输出，计算左右轮的差速，从而调整小车转向。
- `follow_line` : 巡线主函数，负责读取传感器、执行安全检查、计算误差、进行PID控制和差速控制来驱动电机。

```

// APP/app_motor.c
void Motion_Handle(void)
{
    Motion_Get_Speed(&car_data);
}

```

```

    if (g_start_ctrl)
    {
        PID_Calc_Motor(&motor_data);
        Motion_Set_Pwm(motor_data.speed_pwm[0], motor_data.speed_pwm[1],
motor_data.speed_pwm[2], motor_data.speed_pwm[3]);
    }
}

void Motion_Set_Pwm(int16_t Motor_1, int16_t Motor_2, int16_t Motor_3, int16_t
Motor_4)
{
    if (Motor_1 >= -MOTOR_MAX_PULSE && Motor_1 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(Motor_M1, Motor_1);
    }
    //...
    if (Motor_4 >= -MOTOR_MAX_PULSE && Motor_4 <= MOTOR_MAX_PULSE)
    {
        Motor_Set_Pwm(Motor_M4, Motor_4);
    }
}

```

app_motor 模块

- `Motion_Handle`: 电机速度闭环控制的核心任务，在10ms定时中断(TIM6)中周期性执行。调用流程：
 1. 调用 `Motion_Get_Speed` 读取编码器反馈速度，内部完成PID运算
 2. 调用 `Motion_Set_Pwm` 输出PWM控制量到电机驱动
- `Motion_Set_Speed`: 将解算出的四个轮子的目标速度保存到 `motor_data` 结构体中，供PID控制器使用。

```

// APP/PID_Motor.h
#define PID_MOTOR_KP (1.5f)
#define PID_MOTOR_KI (0.08f)
#define PID_MOTOR_KD (0.5f)

```

如果使用的不是亚博售卖的310电机，可以在这里修改电机的PID参数，具体修改数值需要自行试验。